

Empirical Evaluation of Quality and Defects in Software Generated by AI Agents: A Comparative Study under Controlled Requirements

Anonymous (double-anonymous review)

Abstract

Source code generated by large language models (LLMs) has become routine in the software engineering (SE) life-cycle. The literature shows, in isolated fragments, that this code exhibits characteristic defect patterns, frequent security vulnerabilities, technical debt, tests with limited effectiveness, and imperfect reproducibility. No reviewed study, however, combines these dimensions in a single controlled chain: generation of *complete systems* by different *AI agents* from the *same requirement*, under *multiple executions*, evaluated by an *independent oracle* covering functionality, defects, security, maintainability, and reproducibility. This paper presents a controlled, prospective, reproducible experiment filling that gap: four agents (distinct models on the same agent framework, 2× Nemotron, Ling, Mimo) each produced three independent executions of a complete system (task-management REST API, Python/FastAPI/PostgreSQL), totaling 12 deliveries evaluated by an independent 81-test oracle suite plus non-functional checks and a Tambon-taxonomy defect matrix with severity and inter-rater agreement. Results: only 3/12 deliveries were bootable; the 12 Tambon defects were 100% Blocker/Critical; the agents’ own tests detected **0/12** defects whereas the oracle detected **4/4** reachable (RQ6/H3); recurring patterns (missing dependency; *passlib+bcrypt* conflict; migration misconfiguration; “ghost” version) repeat across “free” models, suggesting a common “quality trough”. The study descriptively supports that self-approval by an agent’s own tests is no substitute for independent validation.

Keywords: AI-generated software; coding agents; software quality; software defects; independent oracle; software testing; controlled experiment.

1 Introduction

LLMs now permeate the SE lifecycle, from code and test generation to program repair [1]. Reviews consolidate recurring limitations—hallucinations, security weaknesses, generalization issues, interpretability [2]—and the field shows conflicting results: favorable evidence (e.g., the GitHub Copilot controlled experiment [15]) versus unfavorable (distinct defect profiles at scale [5]; frequent vulnerabilities [18]). The absence of consensus justifies a controlled empirical investigation.

Empirical studies show LLM-generated code has characteristic, recurring defects: Tambon et al. [3] analyzed 333 bugs and proposed a ten-pattern taxonomy; Cotroneo et al. [5] compared 500k+ human/AI samples and found *different defect profiles*; LLM outputs are *non-deterministic* [6]. On testing, LLM-generated suites contain *test smells* [9], have limited detection (29–60% in Defects4J) [10], and coverage/mutation lose reliability when bugs exist [11]; special values (*None*, *inf*, *NaN*) are often omitted [12]. On quality, non-functional aspects are underrated [16] and AI code accumulates *code smells* [17]. On security, 24–30% of snippets are vulnerable [18], all analyzed LLMs generate vulnerabilities [19], and security prompting does not reduce density [20]. On coding agents, reproducibility is limited (68.3% run in clean environments; 13.5× dependency expansion [22]) and agents fail unevenly (commit reversion 0.7–7.6%) [25].

Gap. None of the 25 reviewed studies combines, in one controlled and reproducible experiment, the same specification → different agents → complete systems → ≥3 executions → independent oracle → defects + security + maintainability + reproducibility → statistical analysis. Retrospective studies observe real data; most work with snippets or patches, not complete systems; and few separate the generator from the evaluator.

Research problem. Stated with neutrality: although the literature shows that AI-generated code has characteristic defects and tests with low detection ability, there is no controlled empirical evidence on how different AI agents compare when producing *complete systems* from the *same requirements*, evaluated by an *independent oracle*, under *multiple executions*, covering functional quality, defects, security, maintainability, and reproducibility.

Contributions. (i) A first controlled, prospective, reproducible experiment combining, in the same chain, complete systems + different agents + multiple executions + independent oracle + defects/NFR/testability; (ii) evidence of *recurring defect patterns transferable across “free” models* of the same agent pipeline (“quality trough”); (iii) evidence on *testability*: the agents’ own tests detected no defect (0/12), while the independent oracle detected all reachable ones (4/4).

2 Related Work

Organized by theme (full entries with DOIs in References):

- **LLMs in SE.** [1] largest SLR (395 articles); [2] PRISMA review listing recurring limitations. They supply the RQ vocabulary of this study but do not run controlled experiments on complete-system generation.
- **Defects.** [3] the adopted defect taxonomy. [4] self-correction helps (+29.2%)—a variable kept *out* to preserve control. [5] different profiles AI/human (motivates profile analysis). [6] non-determinism (motivates ≥ 3 executions).
- **LLM-based testing.** [7] map of LLM testing. [8] code+tests for small algorithms (this work advances to complete systems). [?] test smells, limited detection 29–60%, unreliable coverage/mutation in the presence of bugs, omission of special values—support distrusting the AI’s own tests.
- **Quality & security.** [13, 14] supervision and context-dependence (neutrality). [15] Copilot experiment. [16] ISO/IEC 25010 adoption. [17] 89.1% *code smells*. [?] vulnerability frequency and limited prompting gains.
- **Coding agents.** [21] agent model separation. [22] reproducibility/dependency gap. [23] requirement vs. intrinsic error. [24, 25] uneven real behavior.

Position. The review (25 sources) shows 8% generate a complete system, 12% use an independent oracle, and 0% combine complete systems + defects + security (gaps G1–G5). This experiment is positioned precisely in that gap.

3 Methodology

3.1 Position, neutrality, pre-registration

Controlled, prospective, reproducible experiment. Hypotheses, RQs, analysis plan, and the neutrality principle were fixed *before* data collection and consolidated as a pre-registration with temporal anchors in the repository history. **Neutrality (C1):** the experiment does not start from “AI is bad”; if data shows AI equal or better on a dimension, that is a result.

3.2 Experimental design

Between-groups comparison (one group per agent) with intra-group repetitions. **Experimental unit:** a complete generated system. **Treatment:** model identity inside the agent. **Replications:** 3 independent executions

Table 1: Agents (treatments).

Treatment	Model	opcode ID
U	Nemotron 3 Ultra	nemotron-3-ultra-free
L	Nemotron 3.5 Lightning	nemotron-3.5-lightning-free
G	Ling 3.0 Flash	ling-3.0-flash-fin-free
M	Mimo v2.5	mimo-v2.5-free

per agent (non-determinism [6]). **Blinding:** oracle written by an independent QA team in a private, immutable repository, out of the agents’ reach.

Agents (same framework **opcode**, varying only the model):

Versions recorded per execution in manifests; agent-tool comparison lost (mitigated by identical loop).

3.3 Controlled specification

Target: **Task-Management REST API** (Python, FastAPI, PostgreSQL). FR1–FR8: registration/login (hash + JWT); task CRUD; permission rules (owner/admin); filters/pagination; input validation; standardized errors; PostgreSQL with migrations; automated tests. NFRs anchored in ISO/IEC 25010 [16] and CWE/OWASP [?]. Specification given identically to all agents; NFRs informed with behavioral acceptance criteria, *without* solution hints.

3.4 Independent oracle

Private, immutable, validated against golden (non-AI) reference. Layers: functional (pytest, 81 tests); negative (extremes, permissions); security (bandit, pip-audit); structural (ruff, radon); testability (agent’s own tests vs. oracle). Coverage reported but not treated as quality evidence [11]. Validation: native (VM without nested virtualization; PostgreSQL + venv + alembic upgrade head + uvicorn + /health).

3.5 Defect matrix

Each defect: ID; agent+execution; location; category (Tampon 10 patterns [3]); severity (Blocker/Critical/Major/Minor, harmonized with CWE); detection (oracle? agent tests?); evidence. Independent double classification (two raters, Cohen’s κ). Density = defects/KLOC.

3.6 Metrics and statistical analysis

$\alpha = 0.05$. χ^2 only conclusive with expected cells ≥ 5 ; otherwise Fisher exact reported as informative.

3.7 Collection procedure

(1) Freeze specification + oracle; (2) new session per agent, same prompt; (3) ≥ 3 sessions/agent; (4) capture

Table 2: RQs, hypotheses, metrics, and tests.

RQ/H	Metric	Test
RQ1 functional	Pass rate per execution	pytest
RQ2/H1 density	Defects/KLOC	descriptive + Poisson CI
RQ3/H2 categories	Category distribution	χ^2 + Fisher exact
RQ4 severity	% Blocker/Critical	descriptive
RQ5/H4 global	Functional vs NFR	per-dimension
RQ6/H3 testability	Agent tests vs oracle	2×2 Fisher
RQ7 reproducibility	Clean environment run	executability
RQ8 variability	Variance across reps	CV

Table 4: Defect density per agent.

Agent	Defects	KLOC	def/KLOC
ultra	4	4.30	0.93
lightning	4	2.93	1.36
ling	1	1.83	0.55
mimo	3	3.27	0.92

Table 5: Tambon category distribution.

Agent	inc	silly	wrong	non	hall	attr
ultra	4	0	0	0	0	0
lightning	0	1	0	1	1	1
ling	0	0	1	0	0	0
mimo	1	2	0	0	0	0

Table 3: Delivery characterization (oracle execution).

Execution	Tests	Fail	Err	Pass	Boot
lightning/e1-e2	0	0	0	0	no
lightning/e3	81	32	48	1	yes
ling/e1-e3	0	0	0	0	no
mimo/e2	81	21	48	12	yes
mimo/e3	81	21	48	12	yes
ultra/e1-e3	0	0	0	0	no

versions, prompt, log, tests; (5) evaluate with oracle + matrix (raters blind); (6) record evidence. Excluded: pilots, infrastructure findings, NFR/non-bug (classified separately).

3.8 Threats to validity

Oracle data leakage (private/immutable); non-determinism (≥ 3 executions); agent confounding (same prompt/environment); model freeze; low power ($n = 3$); rater bias (κ , double classification); publication bias (pre-registration).

4 Results

12 deliveries (4 agents $\times$ 3 executions); pilots excluded. Figures 1–5 generated by reproducible script.

4.1 Functional quality (RQ1)

Only **3/12 bootable**. Pass rates: 1/81 (lightning), 12/81 (mimo $\times 2$). No delivery passed 100% (Fig. 3).

4.2 Defects and density (RQ2–RQ3)

Lightning highest (1.36), ling lowest (0.55) (Fig. 1). With ≤ 4 defects/agent, H1 descriptive (wide Poisson CI).

Category profile (RQ3):

$\chi^2 = 25.7$, $df = 15$, $p = 0.041$, $V = 0.85$ (informative, not conclusive; cells < 5). Ultra: 100% **incomplete_generation** (Fisher $p = 0.010$) (Fig. 2).

4.3 Severity (RQ4)

All 12 defects **Blocker (8) or Critical (4)** (Fig. 5). Expected, inflated by detection bias (10/12 non-bootable).

4.4 Recurring patterns

Transferable across “free” models—common “quality trough.”

4.5 Testability (RQ6/H3)

Agents’ tests: **0/12 defects detected**. Oracle: **4/4 reachable** (lightning/e3 $\times 2$, mimo/e2, e3). Self-approval $\neq$ independent validation (Fig. 4).

4.6 RQ7–RQ8

Reproducibility: 3/12 ran clean. Inter-rater: $\kappa_{\text{cat}} = 0.54$, $\kappa_{\text{sev}} = 0.91$. Variability: mimo stable (0.15); others qualitative.

4.7 Functional $\neq$ global (RQ5/H4)

Bootable deliveries fail 69–80/81 tests (auth/DB); non-bootable may pass static NFRs. Single index collapses dimensions.

5 Discussion

5.1 Main findings

Density differences descriptive (lightning 1.36; ultra 0.93; mimo 0.92; ling 0.55). Ultra profile most sustainable: 100% **incomplete_generation** ($p = 0.010$). χ^2 points to association ($p = 0.041$) but informative only. H1 neither rejected nor accepted; H2 pointwise support (ultra). Severity inflated by non-bootability. Testability: 0/12 vs 4/4—most important result. Functional $\neq$ global. Recurring patterns extend reproducibility findings [22] and uneven reliability [25] to controlled free-model scenario.

Table 6: Recurring defect patterns.

Pattern	Occ.	Examples
Missing dep.	4 (+1)	ultra e1/e3, mimo e1
<i>passlib+bcrypt</i>	3 (+1)	lightning e3, mimo e2/e3
Alembic misconfig.	1 (+1)	ultra e2
Ghost version	1	lightning e1

Table 7: Agent tests vs oracle ($n = 12$).

	Agent YES	Agent NO	Total
Oracle YES	0	4	4
Oracle NO	0	8	8
Total	0	12	12

5.2 Complementary lab study (non-evidential)

Caveat: models/tasks distinct from formal experiment (chat-LLMs, isolated function). Two models detected 6/6, one 5/6; test generation 5/6, 4/6, 4/6. Illustrates separation of *detecting* and *testing*—consistent with [9, 10].

5.3 Limitations

(1) Low power ($n = 12$); (2) 3/12 bootable; (3) native validation, no Docker; (4) moderate κ_{cat} (0.54); (5) specific models at collection date; (6) Blocker bias; (7) single system/stack.

5.4 Synthesis

Agents produced complete systems with low execution (3/12), concentrated defects, no full oracle conformance. Most sustainable: ultra’s profile. Most general: common quality trough. Results do not support “AI does not work” or “AI replaces QA”; they support that quality fell below the functional minimum in 9/12 executions and self-approval $\neq$ independent validation.

6 Conclusion and Future Work

Controlled, prospective, reproducible experiment combining—first time, as far as review reaches—complete-system generation by different agents from same requirement, multiple executions, independent oracle, simultaneous functionality/defects/security/maintainability/reproducibility/testability. Results: low executability (3/12); quality trough across free models; 0/12 vs 4/4 testability. Future: sample expansion, Docker validation, more models/agents, formal pre-registration, longitudinal study, SLR.

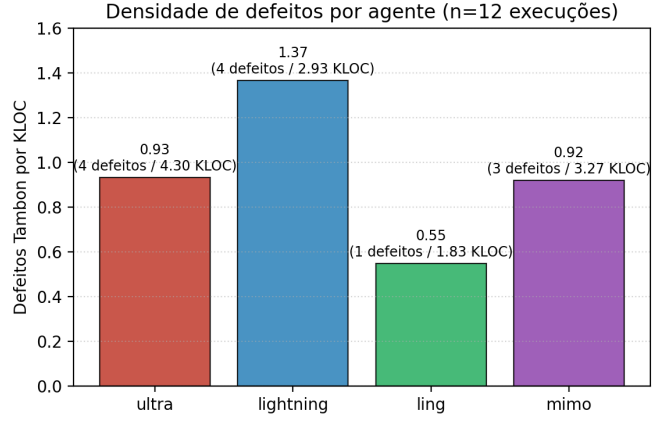


Figure 1: Defect density (defects/KLOC).

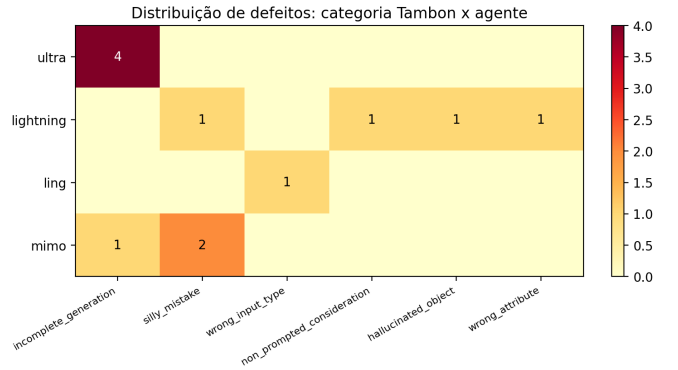


Figure 2: Agent × category heatmap.

References

- [1] Hou, X. et al. Large Language Models for Software Engineering: A Systematic Literature Review. ACM ToSEM, 33(8), 2024. DOI:10.1145/3695988.
- [2] Umama et al. LLM-Based Code Generation: A SLR. IEEE Access, 13:194915–194939, 2025. DOI:10.1109/ACCESS.2025.3631952.
- [3] Tambon, F. et al. Bugs in LLM Generated Code. EMSE, 30:65, 2025. DOI:10.1007/s10664-025-10614-4.
- [4] Dou, S. et al. What’s Wrong with Your Code Generated by LLMs? arXiv:2407.06153, 2024.
- [5] Cotroneo, D. et al. Human-Written vs. AI-Generated Code. IEEE ISSRE 2025, pp. 252–263. DOI:10.1109/ISSRE66568.2025.00035.
- [6] Ouyang, S. et al. Non-Determinism of ChatGPT in Code Generation. ACM ToSEM, 34(2):42, 2025. DOI:10.1145/3697010.
- [7] Wang, J. et al. Software Testing with LLMs. IEEE TSE, 50(4):911–936, 2024. DOI:10.1109/TSE.2024.3368208.

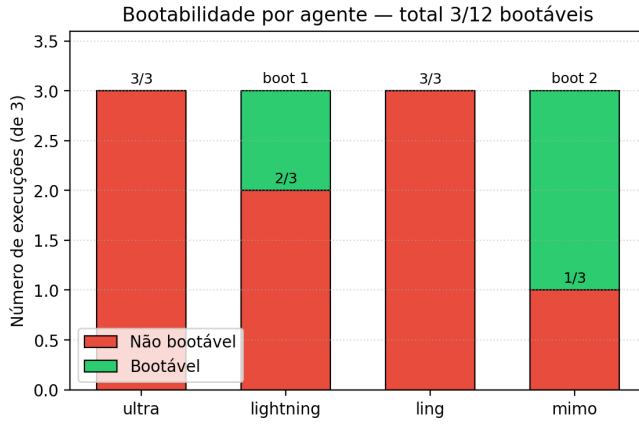


Figure 3: Bootability across deliveries.

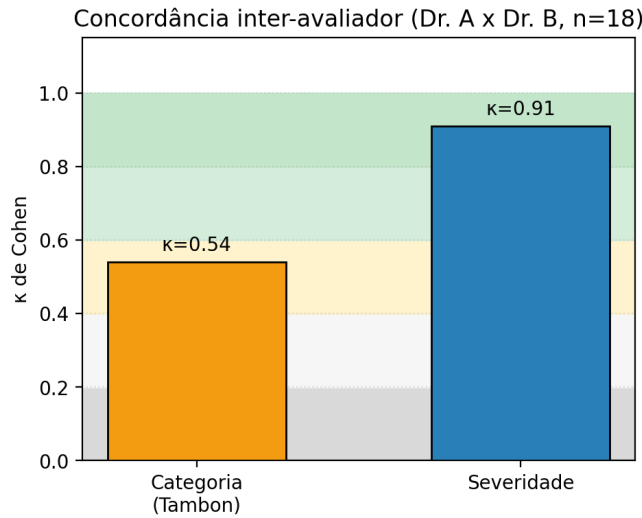


Figure 4: Inter-rater agreement (κ).

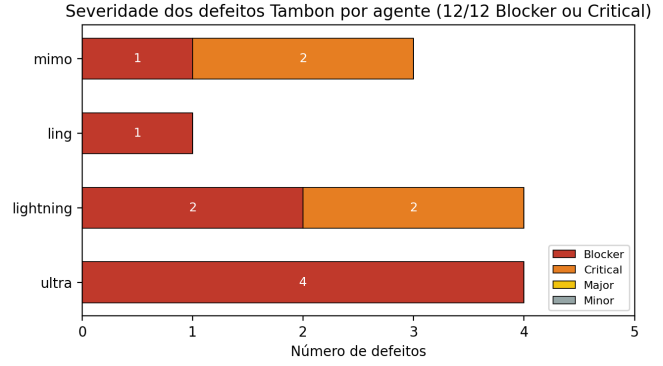


Figure 5: Severity distribution.

- [8] Beer, A. et al. Examination of Code Generated by LLMs. arXiv:2408.16601, 2024.
- [9] Alves, V. A. et al. Detecting Test Smells in Python Tests Generated by LLM. SBES 2024. DOI:10.5753/sbes.2024.3561.
- [10] Yang, L. et al. Evaluation of LLMs in Unit Test Generation. ASE 2024. DOI:10.1145/3691620.3695529.
- [11] Zhao, J. et al. Coverage and Mutation Scores of LLM Test Suites. ISSTA 2026. DOI:10.1145/3832093.
- [12] Walczak, J. et al. Impact of code context on test generation with LLMs. JSS, 237:112834, 2026. DOI:10.1016/j.jss.2026.112834.
- [13] Tosi, D. Quality of Source Code Generated by AI. Future Internet, 16(6):188, 2024. DOI:10.3390/fi16060188.
- [14] Molison, A. S. et al. Is LLM-Generated Code More Maintainable? arXiv:2508.00700, 2025.
- [15] Bauer, J. et al. Does GitHub Copilot improve code quality? GitHub Research, 2024.
- [16] Sun, X. et al. Quality assurance of LLM-generated code. JSS, 238:112885, 2026. DOI:10.1016/j.jss.2026.112885.
- [17] Liu, Y. et al. Debt Behind the AI Boom. arXiv:2603.28592, 2026.
- [18] Fu, Y. et al. Security Weaknesses of Copilot-Generated Code. ACM ToSEM, 34(8), 2025. DOI:10.1145/3716848.
- [19] Morkonda, S. G. et al. Security of LLM-generated Code. arXiv:2605.23091, 2026.
- [20] Kharm, M. et al. LLM-Generated Code Security Across Prompting Methods. arXiv:2605.24298, 2026.
- [21] Horikawa, K. et al. Agentic Refactoring. arXiv:2511.04824, 2025.
- [22] Vangala, B. P. et al. AI-Generated Code Is Not Reproducible (Yet). arXiv:2512.22387. DOI:10.48550/arXiv.2512.22387.
- [23] Belozorov, V. et al. Secure coding with AI. EMSE, 31:93, 2026. DOI:10.1007/s10664-026-10812-8.
- [24] Ehsani, R. et al. Where Do AI Coding Agents Fail? MSR 2026, pp. 807–811. DOI:10.1145/3793302.3793579.
- [25] Oukhay, I. et al. When AI Code Doesn't Stick. MSR 2026, pp. 847–851. DOI:10.1145/3793302.3793587.